pk4proj

Generated by Doxygen 1.10.0

1 Concept Index 1
1.1 Concepts . . . 1

2 Hierarchical Index 3
2.1 Class Hierarchy . . . 3

3 Class Index 5
3.1 Class List . . . 5

4 File Index 7
4.1 File List . . . 7

5 Concept Documentation 9
5.1 AlienLike Concept Reference . . . 9
5.1.1 Concept definition . . . 9
5.1.2 Detailed Description . . . 9

6 Class Documentation 11
6.1 Alien Class Reference . . . 11
6.1.1 Detailed Description . . . 12
6.1.2 Constructor & Destructor Documentation . . . 12
6.1.2.1 Alien() . . . 12
6.1.3 Member Function Documentation . . . 12
6.1.3.1 Draw() . . . 12
6.1.3.2 getRect() . . . 13
6.1.3.3 IsOffScreen() . . . 13
6.1.3.4 Update() . . . 13
6.1.4 Member Data Documentation . . . 13
6.1.4.1 alienImages . . . 13
6.1.4.2 edge . . . 14
6.1.4.3 position . . . 14
6.1.4.4 type . . . 14
6.1.4.5 velocity . . . 14
6.2 alienA Class Reference . . . 14
6.2.1 Detailed Description . . . 16
6.2.2 Constructor & Destructor Documentation . . . 16
6.2.2.1 alienA() . . . 16
6.2.3 Member Function Documentation . . . 16
6.2.3.1 Draw() . . . 16
6.2.3.2 GetImage() . . . 16
6.2.3.3 getRect() . . . 17
6.2.3.4 IsOffScreen() . . . 17
6.2.3.5 LoadImages() . . . 17
6.2.3.6 UnloadImages() . . . 18

6.2.3.7 Update() . . . 18
6.2.4 Member Data Documentation . . . 18
6.2.4.1 alienAImages . . . 18
6.3 alienB Class Reference . . . 18
6.3.1 Detailed Description . . . 20
6.3.2 Constructor & Destructor Documentation . . . 20
6.3.2.1 alienB() . . . 20
6.3.3 Member Function Documentation . . . 20
6.3.3.1 Draw() . . . 20
6.3.3.2 GetImage() . . . 20
6.3.3.3 getRect() . . . 21
6.3.3.4 IsOffScreen() . . . 21
6.3.3.5 LoadImages() . . . 21
6.3.3.6 UnloadImages() . . . 22
6.3.3.7 Update() . . . 22
6.3.4 Member Data Documentation . . . 22
6.3.4.1 alienBImages . . . 22
6.4 alienC Class Reference . . . 22
6.4.1 Detailed Description . . . 24
6.4.2 Constructor & Destructor Documentation . . . 24
6.4.2.1 alienC() . . . 24
6.4.3 Member Function Documentation . . . 24
6.4.3.1 Draw() . . . 24
6.4.3.2 GetImage() . . . 24
6.4.3.3 getRect() . . . 25
6.4.3.4 IsOffScreen() . . . 25
6.4.3.5 LoadImages() . . . 25
6.4.3.6 UnloadImages() . . . 26
6.4.3.7 Update() . . . 26
6.4.4 Member Data Documentation . . . 26
6.4.4.1 alienCImages . . . 26
6.5 Block Class Reference . . . 26
6.5.1 Detailed Description . . . 27
6.5.2 Constructor & Destructor Documentation . . . 27
6.5.2.1 Block() . . . 27
6.5.3 Member Function Documentation . . . 27
6.5.3.1 Draw() . . . 27
6.5.3.2 getRect() . . . 27
6.6 Game Class Reference . . . 28
6.6.1 Detailed Description . . . 29
6.6.2 Constructor & Destructor Documentation . . . 29
6.6.2.1 Game() . . . 29

6.6.2.2 ∼Game() . . . 29
6.6.3 Member Function Documentation . . . 29
6.6.3.1 CreateObstacles() . . . 29
6.6.3.2 Draw() . . . 29
6.6.3.3 HandleInput() . . . 29
6.6.3.4 Reset() . . . 30
6.6.3.5 Update() . . . 30
6.6.4 Member Data Documentation . . . 30
6.6.4.1 alienSpawnInterval . . . 30
6.6.4.2 highscore . . . 30
6.6.4.3 lives . . . 30
6.6.4.4 music . . . 30
6.6.4.5 run . . . 30
6.6.4.6 score . . . 31
6.6.4.7 timeLastAlienFiredA . . . 31
6.6.4.8 timeLastAlienFiredB . . . 31
6.6.4.9 timeLastAlienFiredC . . . 31
6.7 Laser Class Reference . . . 31
6.7.1 Detailed Description . . . 32
6.7.2 Constructor & Destructor Documentation . . . 32
6.7.2.1 Laser() . . . 32
6.7.3 Member Function Documentation . . . 32
6.7.3.1 Draw() . . . 32
6.7.3.2 getRect() . . . 32
6.7.3.3 Update() . . . 33
6.7.4 Member Data Documentation . . . 33
6.7.4.1 active . . . 33
6.7.4.2 position . . . 33
6.7.4.3 velocity . . . 33
6.8 MysteryShip Class Reference . . . 33
6.8.1 Detailed Description . . . 34
6.8.2 Constructor & Destructor Documentation . . . 34
6.8.2.1 MysteryShip() . . . 34
6.8.2.2 ∼MysteryShip() . . . 34
6.8.3 Member Function Documentation . . . 34
6.8.3.1 Draw() . . . 34
6.8.3.2 FireLasers() . . . 34
6.8.3.3 getRect() . . . 35
6.8.3.4 Spawn() . . . 35
6.8.3.5 Update() . . . 35
6.8.4 Member Data Documentation . . . 35
6.8.4.1 alive . . . 35

6.8.4.2 lastFireTime . . . 35
6.8.4.3 mlasers . . . 35
6.9 Obstacle Class Reference . . . 36
6.9.1 Detailed Description . . . 36
6.9.2 Constructor & Destructor Documentation . . . 36
6.9.2.1 Obstacle() . . . 36
6.9.3 Member Function Documentation . . . 37
6.9.3.1 Draw() . . . 37
6.9.4 Member Data Documentation . . . 37
6.9.4.1 blocks . . . 37
6.9.4.2 grid . . . 37
6.9.4.3 position . . . 37
6.10 Spaceship Class Reference . . . 38
6.10.1 Detailed Description . . . 38
6.10.2 Constructor & Destructor Documentation . . . 38
6.10.2.1 Spaceship() . . . 38
6.10.2.2 ∼Spaceship() . . . 39
6.10.3 Member Function Documentation . . . 39
6.10.3.1 Draw() . . . 39
6.10.3.2 FireLaser() . . . 39
6.10.3.3 getRect() . . . 39
6.10.3.4 MoveDown() . . . 39
6.10.3.5 MoveLeft() . . . 39
6.10.3.6 MoveRight() . . . 40
6.10.3.7 MoveUp() . . . 40
6.10.3.8 Reset() . . . 40
6.10.4 Member Data Documentation . . . 40
6.10.4.1 lasers . . . 40
6.10.4.2 movementDirection . . . 40

7 File Documentation **41**
7.1 alien.cpp File Reference . . . 41
7.2 alien.h File Reference . . . 41
7.3 alien.h . . . 41
7.4 alienA.cpp File Reference . . . 42
7.5 alienA.h File Reference . . . 42
7.6 alienA.h . . . 42
7.7 alienB.cpp File Reference . . . 42
7.8 alienB.h File Reference . . . 43
7.9 alienB.h . . . 43
7.10 alienC.cpp File Reference . . . 43
7.11 alienC.h File Reference . . . 43

7.12 alienC.h . . . 44
7.13 block.cpp File Reference . . . 44
7.14 block.h File Reference . . . 44
7.15 block.h . . . 45
7.16 game.cpp File Reference . . . 45
7.17 game.h File Reference . . . 45
7.18 game.h . . . 46
7.19 laser.cpp File Reference . . . 47
7.20 laser.h File Reference . . . 47
7.21 laser.h . . . 48
7.22 main.cpp File Reference . . . 48
7.22.1 Function Documentation . . . 49
7.22.1.1 FormatWithLeadingZeros() . . . 49
7.22.1.2 main() . . . 49
7.23 mysteryship.cpp File Reference . . . 49
7.24 mysteryship.h File Reference . . . 49
7.25 mysteryship.h . . . 50
7.26 obstacle.cpp File Reference . . . 50
7.27 obstacle.h File Reference . . . 50
7.28 obstacle.h . . . 51
7.29 spaceship.cpp File Reference . . . 51
7.30 spaceship.h File Reference . . . 51
7.31 spaceship.h . . . 52

Index **53**

Chapter 1

Concept Index

1.1 Concepts

Here is a list of all concepts with brief descriptions:

AlienLike
Koncept definiujący wymagania dla typów "Alien-podobnych" 9

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Alien 11
- alienA 14
- alienB 18
- alienC 22

Block 26
Game 28
Laser 31
MysteryShip 33
Obstacle 36
Spaceship 38

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Alien
Klasa bazowa dla wszystkich typów obcych w grze 11
alienA
Klasa reprezentująca pierwszy typ obcego (**Alien** (p. 11) A) 14
alienB
Klasa reprezentująca drugi typ obcego (**Alien** (p. 11) B) 18
alienC
Klasa reprezentująca trzeci typ obcego (**Alien** (p. 11) C) 22
Block
Klasa reprezentująca pojedynczy blok, będący częścią przeszkody 26
Game
Klasa zarządzająca logiką i stanem gry 28
Laser
Klasa reprezentująca pojedynczy laser w grze 31
MysteryShip
Klasa reprezentująca tajemniczy statek kosmiczny 33
Obstacle
Klasa reprezentująca przeszkodę złożoną z bloków 36
Spaceship
Klasa reprezentująca statek kosmiczny gracza 38

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

alien.cpp 41
alien.h 41
alienA.cpp 42
alienA.h 42
alienB.cpp 42
alienB.h 43
alienC.cpp 43
alienC.h 43
block.cpp 44
block.h 44
game.cpp 45
game.h 45
laser.cpp 47
laser.h 47
main.cpp 48
mysteryship.cpp 49
mysteryship.h 49
obstacle.cpp 50
obstacle.h 50
spaceship.cpp 51
spaceship.h 51

Chapter 5

Concept Documentation

5.1 AlienLike Concept Reference

Koncept definiujący wymagania dla typów "Alien-podobnych".

```
#include <game.h>
```

5.1.1 Concept definition

```
template<typename T>
concept AlienLike =  requires(T alien) {
    { alien.position } -> std::convertible_to<Vector2>;
    { alien.velocity } -> std::convertible_to<Vector2>;
    { alien.edge } -> std::convertible_to<int>;
    { T::GetImage(alien.edge) } -> std::convertible_to<Texture2D&>;
}
```

5.1.2 Detailed Description

Koncept definiujący wymagania dla typów "Alien-podobnych".

Używany do zapewnienia, że typy używane w szablonach funkcji związanych z obcymi posiadają niezbędne pola i metody. Wymaga pól: `position` (Vector2), `velocity` (Vector2), `edge` (int). Wymaga statycznej metody: `GetImage(int edge)` zwracającej `Texture2D&`.

Template Parameters

T	Typ, który ma zostać sprawdzony pod kątem zgodności z konceptem **AlienLike** (p. 9).

Chapter 6

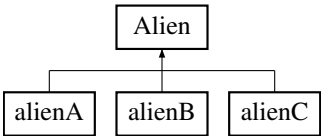
Class Documentation

6.1 Alien Class Reference

Klasa bazowa dla wszystkich typów obcych w grze.

`#include <alien.h>`

Inheritance diagram for Alien:



Public Member Functions

- **Alien** (int **edge**, int **type**, Vector2 **position**, Vector2 **velocity**)

  *Konstruktor klasy **Alien** (p. 11).*
- virtual void **Update** ()=0

  Czysto wirtualna metoda do aktualizacji stanu obcego.
- virtual void **Draw** ()=0

  Czysto wirtualna metoda do rysowania obcego na ekranie.
- virtual bool **IsOffScreen** () const =0

  Czysto wirtualna metoda sprawdzająca, czy obcy jest poza ekranem.
- virtual Rectangle **getRect** () const =0

  Czysto wirtualna metoda zwracająca prostokąt kolizji obcego.

Public Attributes

- int **type**

  Typ obcego.
- int **edge**

  Krawędź, z której obcy się pojawił.
- Vector2 **position**

  Pozycja obcego na ekranie.
- Vector2 **velocity**

  Wektor prędkości obcego.

Static Public Attributes

- static Texture2D **alienImages** [3] = {}

  Statyczna tablica przechowująca tekstury dla różnych typów obcych.

6.1.1 Detailed Description

Klasa bazowa dla wszystkich typów obcych w grze.

- Klasa **Alien** (p. 11) definiuje wspólny interfejs i podstawowe właściwości dla różnych typów obcych. Jest to klasa abstrakcyjna, co oznacza, że nie można tworzyć jej bezpośrednich instancji, a klasy pochodne muszą implementować jej czysto wirtualne metody.

6.1.2 Constructor & Destructor Documentation

6.1.2.1 Alien()

```
Alien::Alien (
            int edge,
            int type,
            Vector2 position,
            Vector2 velocity )
```

Konstruktor klasy **Alien** (p. 11).

Parameters

edge	Krawędź, z której obcy się pojawi (1: lewo, 2: prawo, 3: góra).
type	Typ obcego (może być użyty do rozróżniania zachowań lub wyglądu).
position	Pozycja początkowa obcego (wektor X, Y).
velocity	Prędkość i kierunek ruchu obcego (wektor X, Y).

6.1.3 Member Function Documentation

6.1.3.1 Draw()

```
virtual void Alien::Draw ( )  [pure virtual]
```

Czysto wirtualna metoda do rysowania obcego na ekranie.

- Implementacja tej metody będzie zależeć od konkretnego typu obcego.

Implemented in **alienA** (p. 16), **alienB** (p. 20), and **alienC** (p. 24).

6.1.3.2 getRect()

```
virtual Rectangle Alien::getRect ( ) const  [pure virtual]
```

Czysto wirtualna metoda zwracająca prostokąt kolizji obcego.

Returns

Obiekt Rectangle reprezentujący obszar kolizji obcego.

Implemented in **alienA** (p. 17), **alienB** (p. 21), and **alienC** (p. 25).

6.1.3.3 IsOffScreen()

```
virtual bool Alien::IsOffScreen ( ) const  [pure virtual]
```

Czysto wirtualna metoda sprawdzająca, czy obcy jest poza ekranem.

Returns

true jeśli obcy jest poza granicami ekranu, false w przeciwnym razie.

Implemented in **alienA** (p. 17), **alienB** (p. 21), and **alienC** (p. 25).

6.1.3.4 Update()

```
virtual void Alien::Update ( )  [pure virtual]
```

Czysto wirtualna metoda do aktualizacji stanu obcego.

- Implementacja tej metody będzie zależeć od konkretnego typu obcego.

Implemented in **alienA** (p. 18), **alienB** (p. 22), and **alienC** (p. 26).

6.1.4 Member Data Documentation

6.1.4.1 alienImages

```
Texture2D Alien::alienImages = {}  [static]
```

Statyczna tablica przechowująca tekstury dla różnych typów obcych.

- Dostęp do tekstur powinien być realizowany poprzez klasy pochodne.

6.1.4.2 edge

```
int Alien::edge
```

Krawędź, z której obcy się pojawił.

6.1.4.3 position

```
Vector2 Alien::position
```

Pozycja obcego na ekranie.

6.1.4.4 type

```
int Alien::type
```

Typ obcego.

6.1.4.5 velocity

```
Vector2 Alien::velocity
```

Wektor prędkości obcego.

The documentation for this class was generated from the following files:

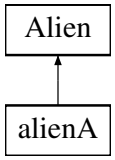
- **alien.h**
- **alien.cpp**

6.2 alienA Class Reference

Klasa reprezentująca pierwszy typ obcego (**Alien** (p. 11) A).

```
#include <alienA.h>
```

Inheritance diagram for alienA:



Public Member Functions

- **alienA** (int **edge**, int **type**, Vector2 **position**, Vector2 **velocity**)

  *Konstruktor klasy **alienA** (p. 14).*
- void **Update** () override

  Aktualizuje pozycję obcego.
- void **Draw** () override

  Rysuje obcego na ekranie.
- bool **IsOffScreen** () const override

  Sprawdza, czy obcy jest poza ekranem.
- Rectangle **getRect** () const override

  Zwraca prostokąt kolizji obcego.

Public Member Functions inherited from Alien

- **Alien** (int **edge**, int **type**, Vector2 **position**, Vector2 **velocity**)

  *Konstruktor klasy **Alien** (p. 11).*

Static Public Member Functions

- static void **LoadImages** ()

  *Statyczna metoda do ładowania wszystkich tekstur dla **Alien** (p. 11) A.*
- static void **UnloadImages** ()

  *Statyczna metoda do rozładowywania wszystkich tekstur dla **Alien** (p. 11) A.*
- static Texture2D & **GetImage** (int **edge**)

  *Statyczna metoda zwracająca referencję do tekstury **Alien** (p. 11) A na podstawie krawędzi.*

Static Public Attributes

- static Texture2D **alienAImages** [3] = {}

  *Statyczna tablica przechowująca tekstury dla **Alien** (p. 11) A.*

Static Public Attributes inherited from Alien

- static Texture2D **alienImages** [3] = {}

  Statyczna tablica przechowująca tekstury dla różnych typów obcych.

Additional Inherited Members

Public Attributes inherited from Alien

- int **type**

  Typ obcego.
- int **edge**

  Krawędź, z której obcy się pojawił.
- Vector2 **position**

  Pozycja obcego na ekranie.
- Vector2 **velocity**

  Wektor prędkości obcego.

6.2.1 Detailed Description

Klasa reprezentująca pierwszy typ obcego (**Alien** (p. 11) A).

- Dziedziczy po klasie **Alien** (p. 11) i implementuje jej czysto wirtualne metody. Obsługuje ładowanie i rozładowywanie własnych tekstur oraz specyficzne zachowanie.

6.2.2 Constructor & Destructor Documentation

6.2.2.1 alienA()

```
alienA::alienA (
            int edge,
            int type,
            Vector2 position,
            Vector2 velocity )
```

Konstruktor klasy **alienA** (p. 14).

Parameters

edge	Krawędź, z której obcy się pojawi (1: lewo, 2: prawo, 3: góra).
type	Typ obcego (dla **alienA** (p. 14) zawsze 0, ale zachowane dla spójności z klasą bazową).
position	Pozycja początkowa obcego (wektor X, Y).
velocity	Prędkość i kierunek ruchu obcego (wektor X, Y).

6.2.3 Member Function Documentation

6.2.3.1 Draw()

```
void alienA::Draw ( )  [override], [virtual]
```

Rysuje obcego na ekranie.

- Implementacja metody wirtualnej z klasy bazowej **Alien** (p. 11). Wykorzystuje teksturę odpowiadającą wartości 'edge' obcego.

Implements **Alien** (p. 12).

6.2.3.2 GetImage()

```
Texture2D & alienA::GetImage (
            int edge )  [static]
```

Statyczna metoda zwracająca referencję do tekstury **Alien** (p. 11) A na podstawie krawędzi.

Parameters

edge	Krawędź (1, 2 lub 3) określająca, którą teksturę zwrócić.

Returns

Referencja do obiektu Texture2D.

6.2.3.3 getRect()

```
Rectangle alienA::getRect ( ) const  [override], [virtual]
```

Zwraca prostokąt kolizji obcego.

Returns

Obiekt Rectangle reprezentujący obszar kolizji obcego.

- Implementacja metody wirtualnej z klasy bazowej **Alien** (p. 11).

Implements **Alien** (p. 12).

6.2.3.4 IsOffScreen()

```
bool alienA::IsOffScreen ( ) const  [override], [virtual]
```

Sprawdza, czy obcy jest poza ekranem.

Returns

true jeśli obcy jest poza granicami ekranu, false w przeciwnym razie.

- Implementacja metody wirtualnej z klasy bazowej **Alien** (p. 11).

Implements **Alien** (p. 13).

6.2.3.5 LoadImages()

```
void alienA::LoadImages ( )  [static]
```

Statyczna metoda do ładowania wszystkich tekstur dla **Alien** (p. 11) A.

- Tekstury są ładowane tylko raz i przechowywane w statycznej tablicy `alienAImages`.

6.2.3.6 UnloadImages()

```
void alienA::UnloadImages ( )  [static]
```

Statyczna metoda do rozładowywania wszystkich tekstur dla **Alien** (p. 11) A.

- Zwolnienie zasobów tekstur.

6.2.3.7 Update()

```
void alienA::Update ( )  [override], [virtual]
```

Aktualizuje pozycję obcego.

- Implementacja metody wirtualnej z klasy bazowej **Alien** (p. 11).

Implements **Alien** (p. 13).

6.2.4 Member Data Documentation

6.2.4.1 alienAImages

```
Texture2D alienA::alienAImages = {}  [static]
```

Statyczna tablica przechowująca tekstury dla **Alien** (p. 11) A.

- Tablica zawiera trzy tekstury odpowiadające różnym krawędziom (edge).

The documentation for this class was generated from the following files:

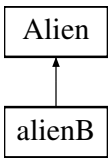
- **alienA.h**
- **alienA.cpp**

6.3 alienB Class Reference

Klasa reprezentująca drugi typ obcego (**Alien** (p. 11) B).

```
#include <alienB.h>
```

Inheritance diagram for alienB:



Public Member Functions

- **alienB** (int **edge**, int **type**, Vector2 **position**, Vector2 **velocity**)

  *Konstruktor klasy **alienB** (p. 18).*
- void **Update** () override

  Aktualizuje pozycję obcego.
- void **Draw** () override

  Rysuje obcego na ekranie.
- bool **IsOffScreen** () const override

  Sprawdza, czy obcy jest poza ekranem.
- Rectangle **getRect** () const override

  Zwraca prostokąt kolizji obcego.

Public Member Functions inherited from Alien

- **Alien** (int **edge**, int **type**, Vector2 **position**, Vector2 **velocity**)

  *Konstruktor klasy **Alien** (p. 11).*

Static Public Member Functions

- static void **LoadImages** ()

  *Statyczna metoda do ładowania wszystkich tekstur dla **Alien** (p. 11) B.*
- static void **UnloadImages** ()

  *Statyczna metoda do rozładowywania wszystkich tekstur dla **Alien** (p. 11) B.*
- static Texture2D & **GetImage** (int **edge**)

  *Statyczna metoda zwracająca referencję do tekstury **Alien** (p. 11) B na podstawie krawędzi.*

Static Public Attributes

- static Texture2D **alienBImages** [3] = {}

  *Statyczna tablica przechowująca tekstury dla **Alien** (p. 11) B.*

Static Public Attributes inherited from Alien

- static Texture2D **alienImages** [3] = {}

  Statyczna tablica przechowująca tekstury dla różnych typów obcych.

Additional Inherited Members

Public Attributes inherited from Alien

- int **type**

  Typ obcego.
- int **edge**

  Krawędź, z której obcy się pojawił.
- Vector2 **position**

  Pozycja obcego na ekranie.
- Vector2 **velocity**

  Wektor prędkości obcego.

6.3.1 Detailed Description

Klasa reprezentująca drugi typ obcego (**Alien** (p. 11) B).

- Dziedziczy po klasie **Alien** (p. 11) i implementuje jej czysto wirtualne metody. Obsługuje ładowanie i rozładowywanie własnych tekstur oraz specyficzne zachowanie.

6.3.2 Constructor & Destructor Documentation

6.3.2.1 alienB()

```
alienB::alienB (
            int edge,
            int type,
            Vector2 position,
            Vector2 velocity )
```

Konstruktor klasy **alienB** (p. 18).

Parameters

edge	Krawędź, z której obcy się pojawi (1: lewo, 2: prawo, 3: góra).
type	Typ obcego (dla **alienB** (p. 18) zawsze 0, ale zachowane dla spójności z klasą bazową).
position	Pozycja początkowa obcego (wektor X, Y).
velocity	Prędkość i kierunek ruchu obcego (wektor X, Y).

6.3.3 Member Function Documentation

6.3.3.1 Draw()

```
void alienB::Draw ( )  [override], [virtual]
```

Rysuje obcego na ekranie.

- Implementacja metody wirtualnej z klasy bazowej **Alien** (p. 11). Wykorzystuje teksturę odpowiadającą wartości 'edge' obcego.

Implements **Alien** (p. 12).

6.3.3.2 GetImage()

```
Texture2D & alienB::GetImage (
            int edge )  [static]
```

Statyczna metoda zwracająca referencję do tekstury **Alien** (p. 11) B na podstawie krawędzi.

Parameters

edge	Krawędź (1, 2 lub 3) określająca, którą teksturę zwrócić.

Returns

Referencja do obiektu Texture2D.

6.3.3.3 getRect()

```
Rectangle alienB::getRect ( ) const  [override], [virtual]
```

Zwraca prostokąt kolizji obcego.

Returns

Obiekt Rectangle reprezentujący obszar kolizji obcego.

- Implementacja metody wirtualnej z klasy bazowej **Alien** (p. 11).

Implements **Alien** (p. 12).

6.3.3.4 IsOffScreen()

```
bool alienB::IsOffScreen ( ) const  [override], [virtual]
```

Sprawdza, czy obcy jest poza ekranem.

Returns

true jeśli obcy jest poza granicami ekranu, false w przeciwnym razie.

- Implementacja metody wirtualnej z klasy bazowej **Alien** (p. 11).

Implements **Alien** (p. 13).

6.3.3.5 LoadImages()

```
void alienB::LoadImages ( )  [static]
```

Statyczna metoda do ładowania wszystkich tekstur dla **Alien** (p. 11) B.

- Tekstury są ładowane tylko raz i przechowywane w statycznej tablicy `alienBImages`.

6.3.3.6 UnloadImages()

```
void alienB::UnloadImages ( )  [static]
```

Statyczna metoda do rozładowywania wszystkich tekstur dla **Alien** (p. 11) B.

- Zwolnienie zasobów tekstur.

6.3.3.7 Update()

```
void alienB::Update ( )  [override], [virtual]
```

Aktualizuje pozycję obcego.

- Implementacja metody wirtualnej z klasy bazowej **Alien** (p. 11).

Implements **Alien** (p. 13).

6.3.4 Member Data Documentation

6.3.4.1 alienBImages

```
Texture2D alienB::alienBImages = {}  [static]
```

Statyczna tablica przechowująca tekstury dla **Alien** (p. 11) B.

- Tablica zawiera trzy tekstury odpowiadające różnym krawędziom (edge).

The documentation for this class was generated from the following files:

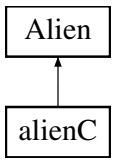
- **alienB.h**
- **alienB.cpp**

6.4 alienC Class Reference

Klasa reprezentująca trzeci typ obcego (**Alien** (p. 11) C).

```
#include <alienC.h>
```

Inheritance diagram for alienC:



Public Member Functions

- **alienC** (int **edge**, int **type**, Vector2 **position**, Vector2 **velocity**)

  *Konstruktor klasy **alienC** (p. 22).*
- void **Update** () override

  Aktualizuje pozycję obcego.
- void **Draw** () override

  Rysuje obcego na ekranie.
- bool **IsOffScreen** () const override

  Sprawdza, czy obcy jest poza ekranem.
- Rectangle **getRect** () const override

  Zwraca prostokąt kolizji obcego.

Public Member Functions inherited from Alien

- **Alien** (int **edge**, int **type**, Vector2 **position**, Vector2 **velocity**)

  *Konstruktor klasy **Alien** (p. 11).*

Static Public Member Functions

- static void **LoadImages** ()

  *Statyczna metoda do ładowania wszystkich tekstur dla **Alien** (p. 11) C.*
- static void **UnloadImages** ()

  *Statyczna metoda do rozładowywania wszystkich tekstur dla **Alien** (p. 11) C.*
- static Texture2D & **GetImage** (int **edge**)

  *Statyczna metoda zwracająca referencję do tekstury **Alien** (p. 11) C na podstawie krawędzi.*

Static Public Attributes

- static Texture2D **alienCImages** [3] = {}

  *Statyczna tablica przechowująca tekstury dla **Alien** (p. 11) C.*

Static Public Attributes inherited from Alien

- static Texture2D **alienImages** [3] = {}

  Statyczna tablica przechowująca tekstury dla różnych typów obcych.

Additional Inherited Members

Public Attributes inherited from Alien

- int **type**

  Typ obcego.
- int **edge**

  Krawędź, z której obcy się pojawił.
- Vector2 **position**

  Pozycja obcego na ekranie.
- Vector2 **velocity**

  Wektor prędkości obcego.

6.4.1 Detailed Description

Klasa reprezentująca trzeci typ obcego (**Alien** (p. 11) C).

- Dziedziczy po klasie **Alien** (p. 11) i implementuje jej czysto wirtualne metody. Obsługuje ładowanie i rozładowywanie własnych tekstur oraz specyficzne zachowanie.

6.4.2 Constructor & Destructor Documentation

6.4.2.1 alienC()

```
alienC::alienC (
            int edge,
            int type,
            Vector2 position,
            Vector2 velocity )
```

Konstruktor klasy **alienC** (p. 22).

Parameters

edge	Krawędź, z której obcy się pojawi (1: lewo, 2: prawo, 3: góra).
type	Typ obcego (dla **alienC** (p. 22) zawsze 0, ale zachowane dla spójności z klasą bazową).
position	Pozycja początkowa obcego (wektor X, Y).
velocity	Prędkość i kierunek ruchu obcego (wektor X, Y).

6.4.3 Member Function Documentation

6.4.3.1 Draw()

```
void alienC::Draw ( )  [override], [virtual]
```

Rysuje obcego na ekranie.

- Implementacja metody wirtualnej z klasy bazowej **Alien** (p. 11). Wykorzystuje teksturę odpowiadającą wartości 'edge' obcego.

Implements **Alien** (p. 12).

6.4.3.2 GetImage()

```
Texture2D & alienC::GetImage (
            int edge )  [static]
```

Statyczna metoda zwracająca referencję do tekstury **Alien** (p. 11) C na podstawie krawędzi.

Parameters

edge	Krawędź (1, 2 lub 3) określająca, którą teksturę zwrócić.

Returns

Referencja do obiektu Texture2D.

6.4.3.3 getRect()

```
Rectangle alienC::getRect ( ) const  [override], [virtual]
```

Zwraca prostokąt kolizji obcego.

Returns

Obiekt Rectangle reprezentujący obszar kolizji obcego.

- Implementacja metody wirtualnej z klasy bazowej **Alien** (p. 11).

Implements **Alien** (p. 12).

6.4.3.4 IsOffScreen()

```
bool alienC::IsOffScreen ( ) const  [override], [virtual]
```

Sprawdza, czy obcy jest poza ekranem.

Returns

true jeśli obcy jest poza granicami ekranu, false w przeciwnym razie.

- Implementacja metody wirtualnej z klasy bazowej **Alien** (p. 11).

Implements **Alien** (p. 13).

6.4.3.5 LoadImages()

```
void alienC::LoadImages ( )  [static]
```

Statyczna metoda do ładowania wszystkich tekstur dla **Alien** (p. 11) C.

- Tekstury są ładowane tylko raz i przechowywane w statycznej tablicy `alienCImages`.

6.4.3.6 UnloadImages()

```
void alienC::UnloadImages ( )  [static]
```

Statyczna metoda do rozładowywania wszystkich tekstur dla **Alien** (p. 11) C.

- Zwolnienie zasobów tekstur.

6.4.3.7 Update()

```
void alienC::Update ( )  [override], [virtual]
```

Aktualizuje pozycję obcego.

- Implementacja metody wirtualnej z klasy bazowej **Alien** (p. 11).

Implements **Alien** (p. 13).

6.4.4 Member Data Documentation

6.4.4.1 alienCImages

```
Texture2D alienC::alienCImages = {}  [static]
```

Statyczna tablica przechowująca tekstury dla **Alien** (p. 11) C.

- Tablica zawiera trzy tekstury odpowiadające różnym krawędziom (edge).

The documentation for this class was generated from the following files:

- **alienC.h**
- **alienC.cpp**

6.5 Block Class Reference

Klasa reprezentująca pojedynczy blok, będący częścią przeszkody.

```
#include <block.h>
```

Public Member Functions

- **Block** (Vector2 position)

  *Konstruktor klasy **Block** (p. 26).*
- void **Draw** ()

  Rysuje blok na ekranie.
- Rectangle **getRect** ()

  Zwraca prostokąt kolizji bloku.

6.5.1 Detailed Description

Klasa reprezentująca pojedynczy blok, będący częścią przeszkody.

- Bloki są podstawowymi elementami składającymi się na przeszkody, które mogą zostać zniszczone przez lasery.

6.5.2 Constructor & Destructor Documentation

6.5.2.1 Block()

```
Block::Block (
            Vector2 position )
```

Konstruktor klasy **Block** (p. 26).

Parameters

position	Pozycja bloku (wektor X, Y).

6.5.3 Member Function Documentation

6.5.3.1 Draw()

```
void Block::Draw ( )
```

Rysuje blok na ekranie.

- Blok jest rysowany jako mały prostokąt w określonym kolorze.

6.5.3.2 getRect()

```
Rectangle Block::getRect ( )
```

Zwraca prostokąt kolizji bloku.

Returns

Obiekt Rectangle reprezentujący obszar kolizji bloku.

The documentation for this class was generated from the following files:

- **block.h**
- **block.cpp**

6.6 Game Class Reference

Klasa zarządzająca logiką i stanem gry.

```
#include <game.h>
```

Public Member Functions

- **Game** ()

  *Konstruktor klasy **Game** (p. 28). Inicjalizuje wszystkie komponenty gry, ładuje zasoby audio i tekstury.*
- ~**Game** ()

  *Destruktor klasy **Game** (p. 28). Zwalnia załadowane zasoby, takie jak dźwięki i tekstury.*
- void **Draw** ()

  Rysuje wszystkie elementy gry na ekranie. Obejmuje rysowanie statku gracza, obcych, laserów, przeszkód, statku-zagadki oraz HUD (wynik, życia).
- void **Update** ()

  Aktualizuje stan gry. Obsługuje ruchy obcych, laserów, statku-zagadki, sprawdzanie kolizji, zarządzanie spawnowaniem obcych i laserów.
- void **HandleInput** ()

  Obsługuje wejście od gracza. Przetwarza naciśnięcia klawiszy do sterowania statkiem kosmicznym gracza i strzelania.
- void **Reset** ()

  Resetuje stan gry do początkowego. Używane po zakończeniu gry w celu rozpoczęcia nowej rundy.
- std::vector< **Obstacle** > **CreateObstacles** ()

  Tworzy przeszkody na planszy.

Public Attributes

- bool **run**

  Flaga wskazująca, czy gra jest uruchomiona.
- int **lives**

  Liczba pozostałych żyć gracza.
- int **score**

  Aktualny wynik gracza.
- int **highscore**

  Najwyższy osiągnięty wynik.
- double **timeLastAlienFiredA**

  Czas ostatniego strzału obcego typu A.
- double **timeLastAlienFiredB**

  Czas ostatniego strzału obcego typu B.
- double **timeLastAlienFiredC**

  Czas ostatniego strzału obcego typu C.
- Music **music**

  Muzyka w tle gry.
- float **alienSpawnInterval**

  Interwał spawnowania obcych w sekundach.

6.6.1 Detailed Description

Klasa zarządzająca logiką i stanem gry.

Klasa **Game** (p. 28) obsługuje inicjalizację, aktualizację i rysowanie wszystkich elementów gry, takich jak statek kosmiczny, obcy, pociski, przeszkody i statek-zagadka. Zarządza również punktacją, życiem gracza, logiką menu oraz zapisem/odczytem najlepszego wyniku.

6.6.2 Constructor & Destructor Documentation

6.6.2.1 Game()

```
Game::Game ( )
```

Konstruktor klasy **Game** (p. 28). Inicjalizuje wszystkie komponenty gry, ładuje zasoby audio i tekstury.

6.6.2.2 ∼Game()

```
Game::∼Game ( )
```

Destruktor klasy **Game** (p. 28). Zwalnia załadowane zasoby, takie jak dźwięki i tekstury.

6.6.3 Member Function Documentation

6.6.3.1 CreateObstacles()

```
std::vector< Obstacle > Game::CreateObstacles ( )
```

Tworzy przeszkody na planszy.

Returns

Wektor obiektów **Obstacle** (p. 36) reprezentujących przeszkody.

6.6.3.2 Draw()

```
void Game::Draw ( )
```

Rysuje wszystkie elementy gry na ekranie. Obejmuje rysowanie statku gracza, obcych, laserów, przeszkód, statku-zagadki oraz HUD (wynik, życia).

6.6.3.3 HandleInput()

```
void Game::HandleInput ( )
```

Obsługuje wejście od gracza. Przetwarza naciśnięcia klawiszy do sterowania statkiem kosmicznym gracza i strzelania.

6.6.3.4 Reset()

```
void Game::Reset ( )
```

Resetuje stan gry do początkowego. Używane po zakończeniu gry w celu rozpoczęcia nowej rundy.

6.6.3.5 Update()

```
void Game::Update ( )
```

Aktualizuje stan gry. Obsługuje ruchy obcych, laserów, statku-zagadki, sprawdzanie kolizji, zarządzanie spawnowaniem obcych i laserów.

6.6.4 Member Data Documentation

6.6.4.1 alienSpawnInterval

```
float Game::alienSpawnInterval
```

Interwał spawnowania obcych w sekundach.

6.6.4.2 highscore

```
int Game::highscore
```

Najwyższy osiągnięty wynik.

6.6.4.3 lives

```
int Game::lives
```

Liczba pozostałych żyć gracza.

6.6.4.4 music

```
Music Game::music
```

Muzyka w tle gry.

6.6.4.5 run

```
bool Game::run
```

Flaga wskazująca, czy gra jest uruchomiona.

6.6.4.6 score

```
int Game::score
```

Aktualny wynik gracza.

6.6.4.7 timeLastAlienFiredA

```
double Game::timeLastAlienFiredA
```

Czas ostatniego strzału obcego typu A.

6.6.4.8 timeLastAlienFiredB

```
double Game::timeLastAlienFiredB
```

Czas ostatniego strzału obcego typu B.

6.6.4.9 timeLastAlienFiredC

```
double Game::timeLastAlienFiredC
```

Czas ostatniego strzału obcego typu C.

The documentation for this class was generated from the following files:

- **game.h**
- **game.cpp**

6.7 Laser Class Reference

Klasa reprezentująca pojedynczy laser w grze.

```
#include <laser.h>
```

Public Member Functions

- **Laser** (Vector2 **position**, Vector2 **velocity**)

  *Konstruktor klasy **Laser** (p. 31).*
- void **Update** ()

  Aktualizuje pozycję lasera i sprawdza, czy nie wyleciał poza ekran.
- void **Draw** ()

  Rysuje laser na ekranie.
- Rectangle **getRect** ()

  Zwraca prostokąt kolizji lasera.

Public Attributes

- Vector2 **position**

  Pozycja lasera na ekranie.
- Vector2 **velocity**

  Wektor prędkości lasera.
- bool **active**

  Flaga określająca, czy laser jest aktywny (widoczny i kolizyjny).

6.7.1 Detailed Description

Klasa reprezentująca pojedynczy laser w grze.

- Lasery mogą być wystrzeliwane zarówno przez gracza, jak i przez obcych.

6.7.2 Constructor & Destructor Documentation

6.7.2.1 Laser()

```
Laser::Laser (
            Vector2 position,
            Vector2 velocity )
```

Konstruktor klasy **Laser** (p. 31).

Parameters

position	Pozycja początkowa lasera.
velocity	Wektor prędkości i kierunku lasera.

6.7.3 Member Function Documentation

6.7.3.1 Draw()

```
void Laser::Draw ( )
```

Rysuje laser na ekranie.

- **Laser** (p. 31) jest rysowany jako prostokąt, którego wymiary zależą od kierunku ruchu.

6.7.3.2 getRect()

```
Rectangle Laser::getRect ( )
```

Zwraca prostokąt kolizji lasera.

Returns

Obiekt Rectangle reprezentujący obszar kolizji lasera.

6.7.3.3 Update()

```
void Laser::Update ( )
```

Aktualizuje pozycję lasera i sprawdza, czy nie wyleciał poza ekran.

6.7.4 Member Data Documentation

6.7.4.1 active

```
bool Laser::active
```

Flaga określająca, czy laser jest aktywny (widoczny i kolizyjny).

6.7.4.2 position

```
Vector2 Laser::position
```

Pozycja lasera na ekranie.

6.7.4.3 velocity

```
Vector2 Laser::velocity
```

Wektor prędkości lasera.

The documentation for this class was generated from the following files:

- **laser.h**
- **laser.cpp**

6.8 MysteryShip Class Reference

Klasa reprezentująca tajemniczy statek kosmiczny.

```
#include <mysteryship.h>
```

Public Member Functions

- **MysteryShip** ()

  *Konstruktor klasy **MysteryShip** (p. 33).*
- ~**MysteryShip** ()

  *Destruktor klasy **MysteryShip** (p. 33).*
- void **Update** ()

  Aktualizuje stan tajemniczego statku (ruch, czy jest aktywny).
- void **Draw** ()

  Rysuje tajemniczy statek na ekranie, jeśli jest aktywny.
- void **Spawn** ()

  Spawnuje tajemniczy statek w losowej pozycji początkowej.
- Rectangle **getRect** ()

  Zwraca prostokąt kolizji tajemniczego statku.
- void **FireLasers** ()

  Powoduje, że tajemniczy statek wystrzeliwuje lasery.

Public Attributes

- bool **alive**

  Flaga określająca, czy tajemniczy statek jest aktywny.
- float **lastFireTime**

  Czas ostatniego strzału tajemniczego statku.
- std::vector< **Laser** > **mlasers**

  Wektor laserów wystrzelonych przez tajemniczy statek.

6.8.1 Detailed Description

Klasa reprezentująca tajemniczy statek kosmiczny.

- Tajemniczy statek pojawia się losowo na górze ekranu, porusza się poziomo i może strzelać laserami.

6.8.2 Constructor & Destructor Documentation

6.8.2.1 MysteryShip()

```
MysteryShip::MysteryShip ( )
```

Konstruktor klasy **MysteryShip** (p. 33).

- Ładuje tekstury i dźwięki, ustawia początkowy stan statku.

6.8.2.2 ~MysteryShip()

```
MysteryShip::~MysteryShip ( )
```

Destruktor klasy **MysteryShip** (p. 33).

- Zwolnienie załadowanych tekstur.

6.8.3 Member Function Documentation

6.8.3.1 Draw()

```
void MysteryShip::Draw ( )
```

Rysuje tajemniczy statek na ekranie, jeśli jest aktywny.

6.8.3.2 FireLasers()

```
void MysteryShip::FireLasers ( )
```

Powoduje, że tajemniczy statek wystrzeliwuje lasery.

6.8.3.3 getRect()

```
Rectangle MysteryShip::getRect ( )
```

Zwraca prostokąt kolizji tajemniczego statku.

Returns

Obiekt Rectangle reprezentujący obszar kolizji statku.

- Zwraca prostokąt o zerowych wymiarach, jeśli statek nie jest aktywny.

6.8.3.4 Spawn()

```
void MysteryShip::Spawn ( )
```

Spawnuje tajemniczy statek w losowej pozycji początkowej.

- Ustawia statek na początku ekranu po lewej lub prawej stronie.

6.8.3.5 Update()

```
void MysteryShip::Update ( )
```

Aktualizuje stan tajemniczego statku (ruch, czy jest aktywny).

6.8.4 Member Data Documentation

6.8.4.1 alive

```
bool MysteryShip::alive
```

Flaga określająca, czy tajemniczy statek jest aktywny.

6.8.4.2 lastFireTime

```
float MysteryShip::lastFireTime
```

Czas ostatniego strzału tajemniczego statku.

6.8.4.3 mlasers

```
std::vector< Laser> MysteryShip::mlasers
```

Wektor laserów wystrzelonych przez tajemniczy statek.

The documentation for this class was generated from the following files:

- **mysteryship.h**
- **mysteryship.cpp**

6.9 Obstacle Class Reference

Klasa reprezentująca przeszkodę złożoną z bloków.

```
#include <obstacle.h>
```

Public Member Functions

- **Obstacle** (Vector2 **position**)

  *Konstruktor klasy **Obstacle** (p. 36).*
- void **Draw** ()

  Rysuje wszystkie aktywne bloki należące do przeszkody.

Public Attributes

- Vector2 **position**

  Pozycja lewego górnego rogu przeszkody.
- std::vector< **Block** > **blocks**

  Wektor bloków tworzących przeszkodę.

Static Public Attributes

- static std::vector< std::vector< int > > **grid**

  Statyczna siatka (mapa) definiująca kształt przeszkody.

6.9.1 Detailed Description

Klasa reprezentująca przeszkodę złożoną z bloków.

- Przeszkody służą jako osłona dla gracza i mogą być niszczone przez lasery.

6.9.2 Constructor & Destructor Documentation

6.9.2.1 Obstacle()

```
Obstacle::Obstacle (
            Vector2 position )
```

Konstruktor klasy **Obstacle** (p. 36).

Parameters

position	Pozycja początkowa lewego górnego rogu przeszkody. • Na podstawie statycznej siatki `grid` tworzy wektor obiektów **Block** (p. 26), które składają się na przeszkodę.

6.9.3 Member Function Documentation

6.9.3.1 Draw()

```
void Obstacle::Draw ( )
```

Rysuje wszystkie aktywne bloki należące do przeszkody.

6.9.4 Member Data Documentation

6.9.4.1 blocks

```
std::vector< Block> Obstacle::blocks
```

Wektor bloków tworzących przeszkodę.

6.9.4.2 grid

```
std::vector< std::vector< int > > Obstacle::grid  [static]
```

Initial value:

```
= {
    {0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0},
    {0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0},
    {0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0},
    {0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0},
    {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1},
    {1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1},
    {0, 1, 1, 1, 1, 0, 0, 0, 1, 1, 1},
    {0, 0, 1, 1, 1, 1, 0, 0, 0, 1, 0},
    {0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0}
}
```

Statyczna siatka (mapa) definiująca kształt przeszkody.

- Wartość 1 oznacza aktywny blok, 0 oznacza pustą przestrzeń.

6.9.4.3 position

```
Vector2 Obstacle::position
```

Pozycja lewego górnego rogu przeszkody.

The documentation for this class was generated from the following files:

- **obstacle.h**
- **obstacle.cpp**

6.10 Spaceship Class Reference

Klasa reprezentująca statek kosmiczny gracza.

```
#include <spaceship.h>
```

Public Member Functions

- **Spaceship** ()

  *Konstruktor klasy **Spaceship** (p. 38).*
- ~**Spaceship** ()

  *Destruktor klasy **Spaceship** (p. 38).*
- void **Draw** ()

  Rysuje statek kosmiczny na ekranie.
- void **MoveLeft** ()

  Przesuwa statek w lewo.
- void **MoveRight** ()

  Przesuwa statek w prawo.
- void **MoveUp** ()

  Przesuwa statek w górę.
- void **MoveDown** ()

  Przesuwa statek w dół.
- void **FireLaser** ()

  Powoduje, że statek wystrzeliwuje laser.
- Rectangle **getRect** ()

  Zwraca prostokąt kolizji statku.
- void **Reset** ()

  Resetuje pozycję statku i usuwa wszystkie aktywne lasery.

Public Attributes

- std::vector< **Laser** > **lasers**

  Wektor laserów wystrzelonych przez statek gracza.
- Vector2 **movementDirection**

  Kierunek ostatniego ruchu statku (używany do określenia kierunku strzału).

6.10.1 Detailed Description

Klasa reprezentująca statek kosmiczny gracza.

- Odpowiada za ruch statku, strzelanie laserami i zarządzanie jego stanem.

6.10.2 Constructor & Destructor Documentation

6.10.2.1 Spaceship()

```
Spaceship::Spaceship ( )
```

Konstruktor klasy **Spaceship** (p. 38).

- Inicjalizuje statek gracza, ładuje tekstury i dźwięki.

6.10.2.2 ~Spaceship()

```
Spaceship::~Spaceship ( )
```

Destruktor klasy **Spaceship** (p. 38).

- Zwolnienie załadowanych tekstur i dźwięków.

6.10.3 Member Function Documentation

6.10.3.1 Draw()

```
void Spaceship::Draw ( )  [inline]
```

Rysuje statek kosmiczny na ekranie.

6.10.3.2 FireLaser()

```
void Spaceship::FireLaser ( )
```

Powoduje, że statek wystrzeliwuje laser.

6.10.3.3 getRect()

```
Rectangle Spaceship::getRect ( )
```

Zwraca prostokąt kolizji statku.

Returns

Obiekt Rectangle reprezentujący obszar kolizji statku.

6.10.3.4 MoveDown()

```
void Spaceship::MoveDown ( )
```

Przesuwa statek w dół.

6.10.3.5 MoveLeft()

```
void Spaceship::MoveLeft ( )
```

Przesuwa statek w lewo.

6.10.3.6 MoveRight()

```
void Spaceship::MoveRight ( )
```

Przesuwa statek w prawo.

6.10.3.7 MoveUp()

```
void Spaceship::MoveUp ( )
```

Przesuwa statek w górę.

6.10.3.8 Reset()

```
void Spaceship::Reset ( )
```

Resetuje pozycję statku i usuwa wszystkie aktywne lasery.

6.10.4 Member Data Documentation

6.10.4.1 lasers

```
std::vector< Laser> Spaceship::lasers
```

Wektor laserów wystrzelonych przez statek gracza.

6.10.4.2 movementDirection

```
Vector2 Spaceship::movementDirection
```

Kierunek ostatniego ruchu statku (używany do określenia kierunku strzału).

The documentation for this class was generated from the following files:

- **spaceship.h**
- **spaceship.cpp**

Chapter 7

File Documentation

7.1 alien.cpp File Reference

```
#include "alien.h"
#include <raylib.h>
```

7.2 alien.h File Reference

```
#include <raylib.h>
```

Classes

- class **Alien**

  Klasa bazowa dla wszystkich typów obcych w grze.

7.3 alien.h

Go to the documentation of this file.

```
#ifndef ALIEN_H
#define ALIEN_H

#include <raylib.h>


class Alien {
public:
    Alien(int edge, int type, Vector2 position, Vector2 velocity);

    virtual void Update() = 0;

    virtual void Draw() = 0;

    virtual bool IsOffScreen() const = 0;

    virtual Rectangle getRect() const = 0;

    static Texture2D alienImages[3];

    int type;
    int edge;

    Vector2 position;
    Vector2 velocity;
private:

};

#endif
```

7.4 alienA.cpp File Reference

```
#include "alienA.h"
#include <raylib.h>
```

7.5 alienA.h File Reference

```
#include "alien.h"
#include <raylib.h>
```

Classes

- class **alienA**

  *Klasa reprezentująca pierwszy typ obcego (**Alien** (p. 11) A).*

7.6 alienA.h

Go to the documentation of this file.

```
00001 #ifndef ALIENA_H
00002 #define ALIENA_H
00003
00004 #include "alien.h"
00005 #include <raylib.h>
00006
00012 class alienA : public Alien
00013 {
00014 public:
00015
00023     alienA(int edge, int type, Vector2 position, Vector2 velocity);
00024
00029     void Update() override;
00030
00036     void Draw() override;
00037
00043     bool IsOffScreen() const override;
00044
00050     Rectangle getRect() const override;
00051
00056     static void LoadImages();
00057
00062     static void UnloadImages();
00063
00068     static Texture2D alienAImages[3];
00069
00075     static Texture2D& GetImage(int edge);
00076
00077
00078 };
00079
00080
00081 #endif
```

7.7 alienB.cpp File Reference

```
#include "alienB.h"
#include <raylib.h>
```

7.8 alienB.h File Reference

```
#include "alien.h"
#include <raylib.h>
```

Classes

- class **alienB**

  *Klasa reprezentująca drugi typ obcego (**Alien** (p. 11) B).*

7.9 alienB.h

Go to the documentation of this file.

```
#ifndef ALIENB_H
#define ALIENB_H

#include "alien.h"
#include <raylib.h>

class alienB : public Alien
{
public:

    alienB(int edge, int type, Vector2 position, Vector2 velocity);

    void Update() override;

    void Draw() override;

    bool IsOffScreen() const override;

    Rectangle getRect() const override;

    static void LoadImages();

    static void UnloadImages();

    static Texture2D alienBImages[3];

    static Texture2D& GetImage(int edge);

};


#endif
```

7.10 alienC.cpp File Reference

```
#include "alienC.h"
#include <raylib.h>
```

7.11 alienC.h File Reference

```
#include "alien.h"
#include <raylib.h>
```

Classes

- class **alienC**

  *Klasa reprezentująca trzeci typ obcego (**Alien** (p. 11) C).*

7.12 alienC.h

Go to the documentation of this file.

```
#ifndef ALIENC_H
#define ALIENC_H

#include "alien.h"
#include <raylib.h>

class alienC : public Alien
{
public:

    alienC(int edge, int type, Vector2 position, Vector2 velocity);

    void Update() override;

    void Draw() override;

    bool IsOffScreen() const override;

    Rectangle getRect() const override;

    static void LoadImages();

    static void UnloadImages();

    static Texture2D alienCImages[3];

    static Texture2D& GetImage(int edge);



};


#endif
```

7.13 block.cpp File Reference

```
#include "block.h"
#include <raylib.h>
```

7.14 block.h File Reference

```
#include "alien.h"
#include <raylib.h>
#include <vector>
#include <iostream>
```

Classes

- class **Block**

  Klasa reprezentująca pojedynczy blok, będący częścią przeszkody.

7.15 block.h

Go to the documentation of this file.

```
#ifndef BLOCK_H
#define BLOCK_H

#include "alien.h"
#include <raylib.h>
#include <vector>
#include <iostream>


class Block {
public:
    Block(Vector2 position);

    void Draw();

    Rectangle getRect();
private:
    Vector2 position;
};
#endif
```

7.16 game.cpp File Reference

```
#include "game.h"
#include <raylib.h>
#include <array>
#include <thread>
#include <mutex>
#include <queue>
#include <chrono>
#include <vector>
#include <regex>
#include <filesystem>
#include <functional>
#include "alien.h"
#include "alienA.h"
#include "spaceship.h"
#include "obstacle.h"
#include "mysteryship.h"
```

7.17 game.h File Reference

```
#include "alien.h"
#include "obstacle.h"
#include "spaceship.h"
#include "mysteryship.h"
#include "laser.h"
#include "alienA.h"
#include "alienB.h"
#include "alienC.h"
#include <raylib.h>
#include <vector>
#include <iostream>
#include <fstream>
#include <chrono>
```

```
#include <concepts>
#include <ranges>
#include <algorithm>
#include <iterator>
#include <memory>
#include <filesystem>
```

Classes

- class **Game**

  Klasa zarządzająca logiką i stanem gry.

Concepts

- concept **AlienLike**

  Koncept definiujący wymagania dla typów "Alien-podobnych".

7.18 game.h

Go to the documentation of this file.

```
00001 #ifndef GAME_H
00002 #define GAME_H
00003
00004 #include "alien.h"
00005 #include "obstacle.h"
00006 #include "spaceship.h"
00007 #include "mysteryship.h"
00008 #include "laser.h"
00009 #include "alienA.h"
00010 #include "alienB.h"
00011 #include "alienC.h"
00012 #include <raylib.h>
00013 #include <vector>
00014 #include <iostream>
00015 #include <fstream>
00016 #include <chrono>
00017 #include <concepts>
00018 #include <ranges>
00019 #include <algorithm>
00020 #include <iterator>
00021 #include <memory>
00022 #include <filesystem> // Dodano to do 'namespace fs = std::filesystem;'
00023
00034 template<typename T>
00035 concept AlienLike = requires(T alien) {
00036     { alien.position } -> std::convertible_to<Vector2>;
00037     { alien.velocity } -> std::convertible_to<Vector2>;
00038     { alien.edge } -> std::convertible_to<int>;
00039     { T::GetImage(alien.edge) } -> std::convertible_to<Texture2D&>;
00040 };
00041
00042 namespace fs = std::filesystem;
00043
00051 class Game {
00052 public:
00057     Game();
00058
00063     ~Game();
00064
00069     void Draw();
00070
00076     void Update();
00077
00082     void HandleInput();
00083
00084     bool run;
00085     int lives;
00086     int score;
```

```
    int highscore;
    double timeLastAlienFiredA;
    double timeLastAlienFiredB;
    double timeLastAlienFiredC;
    Music music;

    void Reset();

    std::vector<Obstacle> CreateObstacles();

    float alienSpawnInterval;

private:
    template <AlienLike AlienType>
    void AlienShootLaserFrom(std::vector<AlienType>& aliens, double& timeLastShot, std::vector<Laser>& laserList, double interval);

    template <AlienLike AlienType>
    void HandleAlienObstacleCollision(std::vector<AlienType>& aliens);

    template<AlienLike AlienType>
    void SpawnAlien(std::vector<AlienType>& container);

    void DeleteInactiveLasers();

    void CheckForCollisions();

    void GameOver();

    void InitGame();

    void checkForHighscore();

    void saveHighscoreToFile(int highscore);

    int loadHighscoreFromFile();

    void DeleteOffScreenAliens();

    Spaceship spaceship;
    std::vector<Obstacle> obstacles;
    std::vector<alienA> aliensA;
    std::vector<alienB> aliensB;
    std::vector<alienC> aliensC;
    std::vector<Alien*> aliensToRemove;
    int currentSpawnType;
    int randomType;
    bool spawning;
    int aliensDirection;
    std::vector<Laser> alienLasers;
    constexpr static float alienLaserShootInterval = 1.0;
    float timeLastAlienFired;
    MysteryShip mysteryship;
    float mysteryShipSpawnInterval;
    float timeLastSpawn;
    Sound explosionSound;
    Sound laserSound;
    float alienSpawnTimer;
};

#endif
```

7.19 laser.cpp File Reference

```
#include "laser.h"
#include "alien.h"
#include <raylib.h>
```

7.20 laser.h File Reference

```
#include "alien.h"
#include <raylib.h>
```

```
#include <vector>
#include <iostream>
```

Classes

- class **Laser**

  Klasa reprezentująca pojedynczy laser w grze.

7.21 laser.h

Go to the documentation of this file.

```
00001 #ifndef LASER_H
00002 #define LASER_H
00003
00004 #include "alien.h"
00005 #include <raylib.h>
00006 #include <vector>
00007 #include <iostream>
00008
00009
00014 class Laser {
00015 public:
00016     Vector2 position;
00017     Vector2 velocity;
00018     bool active;
00019
00025     Laser(Vector2 position, Vector2 velocity);
00026
00030     void Update();
00031
00036     void Draw();
00037
00042     Rectangle getRect();
00043 };
00044 #endif
```

7.22 main.cpp File Reference

```
#include <raylib.h>
#include <vector>
#include <string>
#include <iostream>
#include "game.h"
```

Functions

- std::string **FormatWithLeadingZeros** (int number, int width)

  Formatuje liczbę całkowitą, dodając zera wiodące.
- int **main** ()

  Główna funkcja programu.

7.22.1 Function Documentation

7.22.1.1 FormatWithLeadingZeros()

```
std::string FormatWithLeadingZeros (
            int number,
            int width )
```

Formatuje liczbę całkowitą, dodając zera wiodące.

Funkcja przyjmuje liczbę i pożądaną szerokość ciągu znaków. Jeśli liczba jest krótsza niż `width`, zostaną dodane zera z przodu.

Parameters

number	Liczba do sformatowania.
width	Całkowita szerokość wynikowego ciągu znaków.

Returns

Sformatowany ciąg znaków z zerami wiodącymi.

7.22.1.2 main()

```
int main ( )
```

Główna funkcja programu.

Inicjalizuje okno gry, zasoby audio, ładuje czcionki i tekstury, a następnie zarządza główną pętlą gry, w tym menu początkowym, wyświetlaniem najlepszego wyniku, aktualizacjami gry i rysowaniem. Obsługuje również stany gry (menu, rozgrywka, koniec gry) i nawigację między nimi.

Returns

Kod zakończenia programu.

7.23 mysteryship.cpp File Reference

```
#include "mysteryship.h"
#include "laser.h"
#include <raylib.h>
```

7.24 mysteryship.h File Reference

```
#include "laser.h"
#include "alien.h"
#include <raylib.h>
#include <vector>
#include <iostream>
```

Classes

- class **MysteryShip**

  Klasa reprezentująca tajemniczy statek kosmiczny.

7.25 mysteryship.h

Go to the documentation of this file.

```
#ifndef MYSTERYSHIP_H
#define MYSTERYSHIP_H
#include "laser.h"
#include "alien.h"
#include <raylib.h>
#include <vector>
#include <iostream>



class MysteryShip {
public:
    MysteryShip();

    ~MysteryShip();

    void Update();

    void Draw();

    void Spawn();

    Rectangle getRect();

    bool alive;

    void FireLasers();

    float lastFireTime;
    std::vector<Laser> mlasers;
private:
    Vector2 position;
    Texture2D image;
    int speed;
    Sound laserSound;
};
#endif
```

7.26 obstacle.cpp File Reference

```
#include "obstacle.h"
#include <raylib.h>
```

7.27 obstacle.h File Reference

```
#include "alien.h"
#include "block.h"
#include <raylib.h>
#include <vector>
#include <iostream>
```

Classes

- class **Obstacle**

  Klasa reprezentująca przeszkodę złożoną z bloków.

7.28 obstacle.h

Go to the documentation of this file.

```
#ifndef OBSTACLE_H
#define OBSTACLE_H

#include "alien.h"
#include "block.h"
#include <raylib.h>
#include <vector>
#include <iostream>



class Obstacle {
public:
    Obstacle(Vector2 position);

    void Draw();

    Vector2 position;
    std::vector<Block> blocks;

    static std::vector<std::vector<int» grid;
private:
};
#endif
```

7.29 spaceship.cpp File Reference

```
#include "spaceship.h"
#include <raylib.h>
```

7.30 spaceship.h File Reference

```
#include "alien.h"
#include "laser.h"
#include <raylib.h>
#include <vector>
```

Classes

- class **Spaceship**

  Klasa reprezentująca statek kosmiczny gracza.

7.31 spaceship.h

Go to the documentation of this file.

```
#ifndef SPACESHIP_H
#define SPACESHIP_H

#include "alien.h"
#include "laser.h"
#include <raylib.h>
#include <vector>



class Spaceship {
public:
    Spaceship();

    ~Spaceship();

    void Draw() {
        DrawTextureV(image, position, WHITE);
    }

    void MoveLeft();

    void MoveRight();

    void MoveUp();

    void MoveDown();

    void FireLaser();

    Rectangle getRect();

    void Reset();

    std::vector<Laser> lasers;
    Vector2 movementDirection;

private:
    Texture2D image;
    Vector2 position;
    double lastFireTime;
    Sound laserSound;
};
#endif
```

Index

~Game
    Game, 29
~MysteryShip
    MysteryShip, 34
~Spaceship
    Spaceship, 38

active
    Laser, 33
Alien, 11
    Alien, 12
    alienImages, 13
    Draw, 12
    edge, 13
    getRect, 12
    IsOffScreen, 13
    position, 14
    type, 14
    Update, 13
    velocity, 14
alien.cpp, 41
alien.h, 41
alienA, 14
    alienA, 16
    alienAImages, 18
    Draw, 16
    GetImage, 16
    getRect, 17
    IsOffScreen, 17
    LoadImages, 17
    UnloadImages, 17
    Update, 18
alienA.cpp, 42
alienA.h, 42
alienAImages
    alienA, 18
alienB, 18
    alienB, 20
    alienBImages, 22
    Draw, 20
    GetImage, 20
    getRect, 21
    IsOffScreen, 21
    LoadImages, 21
    UnloadImages, 21
    Update, 22
alienB.cpp, 42
alienB.h, 43
alienBImages
    alienB, 22
alienC, 22
    alienC, 24
    alienCImages, 26
    Draw, 24
    GetImage, 24
    getRect, 25
    IsOffScreen, 25
    LoadImages, 25
    UnloadImages, 25
    Update, 26
alienC.cpp, 43
alienC.h, 43
alienCImages
    alienC, 26
alienImages
    Alien, 13
AlienLike, 9
alienSpawnInterval
    Game, 30
alive
    MysteryShip, 35

Block, 26
    Block, 27
    Draw, 27
    getRect, 27
block.cpp, 44
block.h, 44
blocks
    Obstacle, 37

CreateObstacles
    Game, 29

Draw
    Alien, 12
    alienA, 16
    alienB, 20
    alienC, 24
    Block, 27
    Game, 29
    Laser, 32
    MysteryShip, 34
    Obstacle, 37
    Spaceship, 39

edge
    Alien, 13

FireLaser
    Spaceship, 39

FireLasers
MysteryShip, 34
FormatWithLeadingZeros
main.cpp, 49

Game, 28
~Game, 29
alienSpawnInterval, 30
CreateObstacles, 29
Draw, 29
Game, 29
HandleInput, 29
highscore, 30
lives, 30
music, 30
Reset, 29
run, 30
score, 30
timeLastAlienFiredA, 31
timeLastAlienFiredB, 31
timeLastAlienFiredC, 31
Update, 30
game.cpp, 45
game.h, 45
GetImage
alienA, 16
alienB, 20
alienC, 24
getRect
Alien, 12
alienA, 17
alienB, 21
alienC, 25
Block, 27
Laser, 32
MysteryShip, 34
Spaceship, 39
grid
Obstacle, 37

HandleInput
Game, 29
highscore
Game, 30

IsOffScreen
Alien, 13
alienA, 17
alienB, 21
alienC, 25

Laser, 31
active, 33
Draw, 32
getRect, 32
Laser, 32
position, 33
Update, 32
velocity, 33
laser.cpp, 47
laser.h, 47
lasers
Spaceship, 40
lastFireTime
MysteryShip, 35
lives
Game, 30
LoadImages
alienA, 17
alienB, 21
alienC, 25

main
main.cpp, 49
main.cpp, 48
FormatWithLeadingZeros, 49
main, 49
mlasers
MysteryShip, 35
MoveDown
Spaceship, 39
MoveLeft
Spaceship, 39
movementDirection
Spaceship, 40
MoveRight
Spaceship, 39
MoveUp
Spaceship, 40
music
Game, 30
MysteryShip, 33
~MysteryShip, 34
alive, 35
Draw, 34
FireLasers, 34
getRect, 34
lastFireTime, 35
mlasers, 35
MysteryShip, 34
Spawn, 35
Update, 35
mysteryship.cpp, 49
mysteryship.h, 49

Obstacle, 36
blocks, 37
Draw, 37
grid, 37
Obstacle, 36
position, 37
obstacle.cpp, 50
obstacle.h, 50

position
Alien, 14
Laser, 33
Obstacle, 37

Reset
Game, 29
Spaceship, 40
run
Game, 30

score
Game, 30
Spaceship, 38
~Spaceship, 38
Draw, 39
FireLaser, 39
getRect, 39
lasers, 40
MoveDown, 39
MoveLeft, 39
movementDirection, 40
MoveRight, 39
MoveUp, 40
Reset, 40
Spaceship, 38
spaceship.cpp, 51
spaceship.h, 51
Spawn
MysteryShip, 35

timeLastAlienFiredA
Game, 31
timeLastAlienFiredB
Game, 31
timeLastAlienFiredC
Game, 31
type
Alien, 14

UnloadImages
alienA, 17
alienB, 21
alienC, 25
Update
Alien, 13
alienA, 18
alienB, 22
alienC, 26
Game, 30
Laser, 32
MysteryShip, 35

velocity
Alien, 14
Laser, 33